ECEN 676: Advanced Computer Architecture
Department of Electrical and Computer Engineering
Texas A&M University

Assignment #1

Due Feb 27th, 2026

WARNING THIS HOMEWORK WILL REQUIRE SIGNIFICANT SIMULATION TIME, START EARLY!

Expect that this homework will take about 450-500 core-hours of compute time and plan accordingly (ie. run multiple jobs in parallel as much as possible and start early).

Overview

The purpose of this homework is to familiarize you with trace-driven simulation, an important tool for architecture exploration. You will use Champsim, a simulator developed by the CAMSIN research group at A&M. You will also explore the performance impact of different branch predictors in a CMP architecture. The source code for Champsim is found at:

`https://github.com/ChampSim/ChampSim`

In this homework, you will simulate 13 of the SPEC2017 benchmarks with multiple branch prediction unit configurations. Execution time and misses per thousand instructions (MPKI) will be recorded and analyzed. The homework is written assuming you will run the simulations on the `olympus.ece.tamu.edu` cluster. To use this server, you will need an ECE Unix account. All students registered in the class should automatically receive an ECE Unix account.

Homework Procedure

For this assignment we will use x86 traces with Champsim's (Out-of-Order) CPU model and a 3-level non-inclusive cache hierarchy. We will run a small subset of benchmarks from the SPEC2017 multi-processor benchmark suite. Traces have been created from recording the behavior of these benchmarks at regions of interest.

You may work together with other students in your class on this assignment but each of you is required to write your own:

1. As the first part of your assignment, define these terms (your HW1 report should have the following definitions):
   - Non-Inclusive Cache
   - Trace-driven simulation
2. Set up your working directory:
   (a) Login to the head node `olympus.ece.tamu.edu` using SSH.
   (b) From the head node start an interactive session on one of the compute nodes with the following command:

   ```
   load-ecen-676
   ```

   This command launches a subshell, and will need to be rerun each time you exit the subshell and each time you log in.
   (c) Make a working directory for yourself.
   (d) `cd` into your working directory and use `git` to download the source code from github:

   ```
   git clone https://github.com/ChampSim/ChampSim.git
   cd ChampSim
   ```

   Note: this stage may seem to hang, give it time to finish.
   (e) Configure the Champsim simulator with a basic configuration file. This configuration file specifies that all cores should use the "bimodal" branch predictor, and leaves all remaining options to the defaults, which include a single core containing a bimodal branch predictor, no L1, L2 or LLC prefetcher, and LRU replacement.

   ```
   cp ~pgratz/e676_example_config.json my_config.json
   ```

   (f) From here you should proceed with the instructions given in the ChampSim repository for downloading dependencies and building the Champsim simulator. When you reach the config.sh step be sure to use the `my_confg.json` file you copied over from `~pgratz`.

(g) Because these simulations can take hours to run, you will need to make use of the Slurm batch job scheduler on the cluster to submit the jobs non–interactively. You are going to schedule the jobs in a script which will execute the simulations serially. Copy an example job script into the ChampSim directory.

```
cp ~pgratz/e676_job_submit.sh job_submit.sh
```

(h) Edit the example script for your use. Type your email id in the third line for the mail-user parameter. This will ensure that you will get an email once the jobs mentioned in the script have completed or failed. Set the time in the fifth line to 12:00:00. This indicates that the script will run for 12 hours and terminates irrespective of whether the jobs specified in the script have completed/failed by then.

(i) The line with echo is suggested to include so that you will leave in the log file an indicator to know if the command you will be executing in the following line fails. If the command does fail, then whatever is enclosed in double quotations after echo will be shown in the log file which will appear in the ChampSim directory if the command fails. This will help you locate the command which has failed.

(j) The command in the second line simulates the benchmark by running ChampSim on the `602.gcc_s-734B.champsimtrace.xz` benchmark. The simulation is first warmed up by 200 million instructions so that caches have a lower compulsory, or cold, miss rate. The simulation is then executed for 1 billion instructions while recording information on the branch predictor's accuracy and the cache's MPKI. The resulting output is captured and written to the specified file at the end (after the "$>$" character). More information on the SPEC2017 benchmark suite can be found at the SPEC2017 website (https://www.spec.org/cpu2017/).

(k) Include few more commands in the script similar to the second line with different benchmarks from step 4. Do not forget to also include the echo line before every command you include, and do not forget to specify a new output file, or the old one will be overwritten. Since you are setting the time to 12:00:00, include between 1-4 commands (excluding the echo lines) in a single script file because each benchmark takes somewhere between 2 hours to 7 hours (refer to Figure 1). **In any case, do not change the time to more than 12:00:00 hrs**.

(l) After you have created the script, save it and exit. From the ChampSim directory, execute the following command:

```
sbatch job_submit.sh
```

(m) This command will start executing the script you have created and send you an email upon job completion or failure. You could also check the status of your script using the following:

```
squeue -u <your NetID>
```

If the status of your job is R, it is still running, if it is C, its completed or if it is Q, it is queued and will start eventually.

(n) After the job has completed, examine the output file (in the example above, `602.gcc_s-734B.champsimtrace.txt`).
The output file will contain a variety of information about the cache accesses and branch prediction accuracy.

(o) Create another script file for the remaining benchmarks. You will be using multiple scripts to complete your simulations in parallel.

3. Modify the branch predictor configuration and re-simulate. The branch predictor configuration can be changed by modifying the source files in `branch/`. Open the source files in the "bimodal" directory and read through them. Once you have a general understanding of how the predictor works do the following:

   (a) Change the branch predictor table size to half its original size.

   (b) Rebuild the simulator executable as described above after changing the branch configuration.

   (c) After rebuilding the simulator, create a script and run it. Make sure to save the output of the simulation in new output files, or the results may be overwritten.

   (d) Examine the new result files for the smaller bimodal prediction simulation and compare them to the default configuration.

4. The following are all the benchmarks you need to simulate. There are 13 benchmarks in total.

   - 602.gcc_s-734B.champsimtrace.xz
   - 603.bwaves_s-3699B.champsimtrace.xz
   - 605.mcf_s-665B.champsimtrace.xz
   - 620.omnetpp_s-874B.champsimtrace.xz
   - 623.xalancbmk_s-700B.champsimtrace.xz
   - 625.x264_s-18B.champsimtrace.xz
   - 627.cam4_s-573B.champsimtrace.xz
   - 631.deepsjeng_s-928B.champsimtrace.xz
   - 638.imagick_s-10316B.champsimtrace.xz
   - 641.leela_s-800B.champsimtrace.xz
   - 644.nab_s-5853B.champsimtrace.xz
   - 648.exchange2_s-1699B.champsimtrace.xz
   - 654.roms_s-842B.champsimtrace.xz

5. Using the steps outlined above, measure the instructions-per-cycle (IPC), branch prediction MPKI, and branch prediction accuracy of all the benchmarks with the following branch predictor configurations, leaving all the other entries in the file at the default settings. To use the Perceptron or G-Share predictors, modify your configuration file to specify "`perceptron`" or "`gshare`", respectively.

   - A simple, bimodal branch predictor with the following different configurations:
     - Bimodal Table Size 16384
     - Bimodal Table Size 8192 (prime should be set to 8191)
     - Bimodal Table Size 16384 and Max Counter Size 1
   - A Perceptron branch predictor with the following configurations:
     - Perceptron history: 24, Number of perceptrons: 163, Perceptron Bits: 8
     - Perceptron history: 24, Number of perceptrons: 81, Perceptron Bits: 8
     - Perceptron history: 24, Number of perceptrons: 163, Perceptron Bits: 4
   - A G-Share branch predictor with the following configurations:
     - Global History Length: 14, History Table Size: 16384
     - Global History Length: 14, History Table Size: 8192
     - Global History Length: 7, History Table Size: 16384

6. For more information on the G-Share (http://www.hpl.hp.com/techreports/Compaq-DEC/WRL-TN-36.pdf) and Perceptron (https://ieeexplore.ieee.org/document/903263) predictors, review the papers that originally featured these predictors. Examine the output files for each of the simulation runs and provide a write-up, discussing the IPC and the branch mispredictions per thousand instructions (MPKI), i.e. (# of mispredictions)/(total # instruction/1000), for each run.

7. A few tips:

   - **Start early!** Olympus supports a maximum of 6 parallel jobs per user. If you parallelize efficiently, the best-case total runtime will be approximately 3–4 days, though it may take longer in practice.
   - If you are familiar with the TAMU High Performance Research Computing (HPRC) system (*Faster* or *Grace*), feel free to use it for this assignment.
   - The graph below provides approximate runtime for each benchmark (per branch predictor configuration) to help you plan your batch submissions and manage parallel execution effectively:

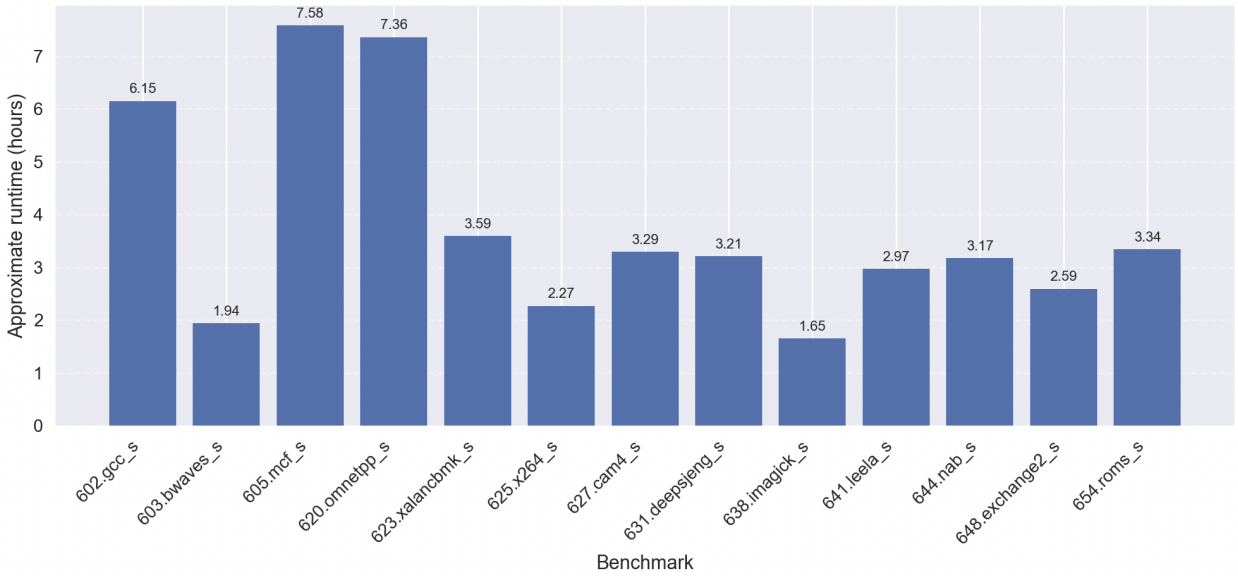

Figure 1: Approx runtime per benchmark, per configuration

Homework Deliverables

You will turn in a short report (**PDF** *only*) into Canvas containing the following:

15 Points Definition for the terms mentioned on page two of this assignment.

15 Points A short description of what each application (13 benchmarks) is (ie. what is its function outside of being a benchmark). You will have to do a bit of googling for this, the SPEC website should help.

30 Points For each of the three configurations, tell the IPC of each benchmark, as well as the branch mis-predictions per thousand instructions (MPKI), i.e. (# of mis predictions)/(total # instruction/1000) (arithmetic mean of IPC and geomean of MPKI across all cores for each experiment).

40 Points Write a 1-2 paragraph analysis of the results found, discussing the impact of branch predictors perf- ormance on each benchmark. What is the complexity/performance trade-off? You should convey some understanding of the results shown.

Note: Getting familiarized with slurm scheduling and writing scripts will be very useful for your project and following homeworks. You will find many tutorials about slurm scheduling and writing scripts online. Take this opportunity to learn these too.